Computer Organization and Architecture

Review Guide · Sessions 1–7

Use this guide to connect the main ideas, formulas, and calculation methods from the first seven sessions.

Focus on understanding what each component does, how information moves through the system, and how to trace or calculate a result step by step.

1. Computer Organization and Architecture

A computer can be viewed as three core parts: CPU, memory, and input/output (I/O). The CPU executes instructions, memory stores programs and data, and I/O connects the computer to the outside world.

Concept	Meaning
Computer Architecture	Programmer-visible design: instruction set, registers, data types, addressing modes.
Computer Organization	How the architecture is implemented: buses, control signals, memory technology, datapaths.
Stored-program concept	Instructions and data are stored in memory.
Hardware–software trade-off	Hardware is usually faster and more specialized; software is easier to change and more flexible.
Von Neumann model	CPU, memory, and I/O with instructions and data sharing the same memory system.
Von Neumann bottleneck	CPU–memory communication can limit performance.
Parallel processing	Multiple operations or instruction streams can proceed at the same time.

Moore's Law

Moore's Law is a historical observation that transistor counts on integrated circuits increased rapidly, roughly doubling about every two years. Transistor growth and performance growth are related but are not the same thing.

If a model assumes performance doubles every d years, use:

Performance factor = 2^(years / d)

Execution time is inversely proportional to performance

2. CPU, Registers, Buses, and Clock

Component	Role
ALU	Performs arithmetic and logical operations.
Registers	Very fast storage inside the CPU.
Control Unit	Decodes instructions and generates control signals.
Datapath	Moves and processes data among registers, the ALU, and internal buses.

System Bus

Bus	Carries
Data bus	The actual data being transferred.
Address bus	The location in memory or I/O being accessed.
Control bus	Signals such as READ, WRITE, and timing/control information.

Clock

The clock synchronizes CPU operations. A higher clock rate means more cycles per second, but one instruction may require several cycles.

Clock frequency: f = 1 / T

Clock-cycle time: T = 1 / f

Unit	Meaning
1 GHz	10^9 cycles per second
1 MHz	10^6 cycles per second
1 ns	10^-9 seconds

3. Memory Addressing

The number of address bits depends on the number of individually addressable locations.

Address bits = ceil(log2(number of addressable locations))

If the number of locations is exactly 2^n, then n address bits are required.

Byte-Addressable vs. Word-Addressable Memory

Type	Meaning
Byte-addressable	Each byte has its own address.
Word-addressable	Each word has its own address.

For a memory with W words and b bits per word (where b is a multiple of 8):

Bytes per word = b / 8

Byte-addressable locations = W × (b / 8)

Address bits = ceil(log2(byte-addressable locations))

Useful Powers of Two

Memory quantity	Power of 2
1K	2^10
4K	2^12

64K	2^16
1M	2^20
4M	2^22
1G	2^30

4. Binary, Hexadecimal, and Two's Complement

Unsigned Binary

Unsigned value = Σ(bit × 2^position)

n-bit unsigned range = 0 to 2^n − 1

Hexadecimal

One hexadecimal digit represents exactly 4 binary bits. One byte is 8 bits, so one byte is represented by 2 hexadecimal digits.

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Binary → hexadecimal: group the bits into groups of four from the right. Hexadecimal → binary: replace each hex digit with its four-bit pattern.

Example: $1011\ 0110_2 = B6_{16}$

Two's Complement

n-bit two's-complement range = −2^(n−1) to 2^(n−1) − 1

To represent a negative value: write the positive value in binary, invert every bit, then add 1.

Step	Example for −10 in 8 bits
Write +10	00001010
Invert all bits	11110101
Add 1	11110110

Quick sign check: in two's complement, MSB = 0 means non-negative; MSB = 1 means negative.

For a negative n-bit pattern: signed value = unsigned value – 2^n

Character Representation

Characters are stored as numeric codes. ASCII assigns numbers to characters; standard ASCII uses 7 bits and is commonly stored in an 8-bit byte. The same bit pattern can mean different things depending on how it is interpreted—for example, characters, an integer, or part of an instruction.

Endianness

Byte order	Lowest memory address contains
Big endian	Most significant byte
Little endian	Least significant byte

Endianness changes the order in which bytes of a multi-byte value are stored; it does not change the bits inside each byte.

5. MARIE: A Simple Computer

MARIE is a teaching architecture that makes instruction execution easy to trace. It is a stored-program, word-addressable machine.

MARIE property	Value
Main memory	4K words = 4096 locations = 2^12
Address size	12 bits
Word size	16 bits
Instruction size	16 bits
Instruction format	4-bit opcode + 12-bit address

Important Registers

Register	Purpose
AC	Accumulator: holds data and arithmetic results.
MAR	Memory Address Register: holds the address being accessed.
MBR	Memory Buffer Register: holds data moving to or from memory.
PC	Program Counter: holds the address of the next instruction.
IR	Instruction Register: holds the current instruction.
InREG	Input register.
OutREG	Output register.

X = an address | M[X] = the value stored at address X

Core MARIE Instructions

Opcode	Instruction	Meaning

1	LOAD X	AC ← M[X]
2	STORE X	M[X] ← AC
3	ADD X	AC ← AC + M[X]
4	SUBT X	AC ← AC − M[X]
5	INPUT	AC ← input
6	OUTPUT	output AC
7	HALT	stop execution

Because the opcode is one hexadecimal digit and the address is three hexadecimal digits, a basic MARIE instruction can be written as:

Machine code = opcode digit + 3-digit hexadecimal address

Register Transfer Notation (RTN)

LOAD X

MAR ← X | MBR ← M[MAR] | AC ← MBR

ADD X

MAR ← X | MBR ← M[MAR] | AC ← AC + MBR

STORE X

MAR ← X | MBR ← AC | M[MAR] ← MBR

Think about where the address goes, where the data goes, and whether the ALU is needed.

Fetch–Decode–Execute Cycle

Stage	Core idea
Fetch	MAR ← PC; MBR ← M[MAR]; IR ← MBR; PC ← PC + 1.
Decode	The control unit separates the instruction into opcode and operand/address fields: opcode = IR[15:12], address = IR[11:0].
Execute	The CPU performs the operation. The exact register transfers depend on the instruction (for example LOAD, ADD, STORE, INPUT, or OUTPUT).

MARIE instruction: IR[15:12] = opcode | IR[11:0] = address

Full fetch: MAR ← PC | MBR ← M[MAR] | IR ← MBR | PC ← PC + 1

Reading MARIE Machine Instructions

A four-digit hexadecimal MARIE instruction is read as one opcode digit followed by a three-digit address. When tracing the CPU at register-transfer level, include the MBR as the buffer between memory and CPU registers.

Machine word	Opcode	Address	Interpretation

130C	1	30C	LOAD 30C
2450	2	450	STORE 450
33A1	3	3A1	ADD 3A1

Assembler and Symbol Table

An assembler translates assembly language into machine code. A symbol table maps each label to its memory address.

Symbol table: label → memory address

When tracing a program, keep track of the AC, the PC, and any memory location changed by STORE or other memory-writing instructions.

6. Instruction Set Architecture (ISA)

An ISA defines the operations a CPU understands and how instructions identify their operands.

Design choice	Examples
Instruction length	Fixed length or variable length
Internal storage	Stack, accumulator, general-purpose registers
Number of explicit operands	0-address, 1-address, 2-address, 3-address
Operand location	Immediate value, register, memory
Addressing	Direct, indirect, indexed, and other modes

Instruction Categories

Category	Examples
Data movement	LOAD, STORE, MOVE, PUSH, POP
Arithmetic	ADD, SUBTRACT, MULTIPLY, DIVIDE
Logic / bit manipulation	AND, OR, XOR, NOT, SHIFT, ROTATE
Control flow	JUMP, BRANCH
I/O	INPUT, OUTPUT

Addressing Modes

Mode	How the operand is found
Immediate	The value is inside the instruction.
Register	The value is in a register.
Direct	Effective address EA = A.
Indirect	Effective address EA = M[A].
Indexed	An index value is added to a base/address field.

Indexed addressing: EA = Base + Index

Array addressing: EA = Base + (Index × Element Size)

7. CPU Performance and Pipelining

CPU Performance Equations

CPU cycles = Instruction Count × CPI

CPU time = Instruction Count × CPI × Clock-Cycle Time

CPU time = (Instruction Count × CPI) / Clock Rate

CPI = (CPU time × Clock Rate) / Instruction Count

Performance ∝ 1 / Execution Time

Speedup = Old execution time / New execution time

A faster clock alone does not guarantee a faster program. Instruction count, CPI, clock rate, memory behavior, and the program itself all matter.

Instruction Pipelining

Pipelining overlaps stages of different instructions. It improves throughput by keeping different pipeline stages busy at the same time. It does not make one individual instruction complete instantly.

Let n = number of instructions, k = number of pipeline stages, and t = pipeline clock-cycle time.

Ideal pipeline time = (k + n − 1) × t

Non-pipelined time = n × time per complete instruction

Pipeline speedup = Non-pipelined time / Pipelined time

If a non-pipelined instruction consists of k equal stages, each taking t:

Speedup = (n × k) / (k + n − 1)

For a very large number of instructions, ideal speedup approaches the number of stages k.

Pipeline Hazards

Hazard	What happens	Typical response
Structural	Two instructions need the same hardware resource.	Stall or provide additional hardware.
Data	An instruction needs a result that is not ready.	Stall or forwarding/bypassing.
Control	The next instruction depends on a branch or jump.	Stall, flush, or branch prediction.

With s stall cycles: Pipeline time = (k + n − 1 + s) × t

8. Memory Hierarchy

The memory hierarchy balances speed, cost, and capacity. The closer memory is to the CPU, the faster and more expensive per bit it tends to be.

Registers → Cache → Main Memory → Secondary Storage

Moving downward in the hierarchy	General trend
Capacity	Increases
Cost per bit	Decreases
Latency	Increases
Speed	Decreases

Latency and Bandwidth

Term	Meaning
Latency	Time needed to complete a memory access.
Bandwidth	Amount of data transferred per unit time.

SRAM and DRAM

SRAM	DRAM
Faster	Slower
More expensive	Less expensive
No periodic refresh	Requires refresh
Commonly used for cache	Commonly used for main memory

Locality

Type	Meaning
Temporal locality	Recently accessed data is likely to be accessed again soon.
Spatial locality	Data near a recently accessed location is likely to be accessed soon.

9. Cache Memory

A cache stores copies of data from main memory so the CPU can access frequently needed information more quickly.

Hit rate = Hits / Total memory accesses

Miss rate = 1 − Hit rate

Cache Mapping

Mapping	Placement rule
Direct mapped	Each main-memory block has exactly one possible cache line.
Fully associative	A main-memory block may be placed in any cache line.
Set associative	A block maps to one set and may use any line within that set.

Direct-Mapped Cache Calculations

Let main memory contain 2^m addressable bytes, the cache have 2^s lines, and the block size be 2^b bytes.

Address format = Tag | Index | Offset

Offset bits = log2(Block Size in bytes)

Index bits = log2(Number of cache lines)

Tag bits = Address bits − Index bits − Offset bits

Number of main-memory blocks = Main Memory Size / Block Size

Memory block number = floor(Address / Block Size)

Cache index = Memory block number mod Number of cache lines

Block offset = Address mod Block Size

Tag value = floor(Memory block number / Number of cache lines)

Binary shortcut: the lowest b address bits are the offset, the next s bits are the cache index, and all remaining upper bits are the tag.

Set-Associative Cache Calculations

For an N-way set-associative cache:

Number of sets = Number of cache lines / N

Set index bits = log2(Number of sets)

Set number = Memory block number mod Number of sets

Tag bits = Address bits − Set Index bits − Offset bits

Fully Associative Cache

There is no index field because a block may be placed anywhere in the cache.

Tag bits = Address bits − Offset bits

Replacement Policies

Policy	Meaning
FIFO	Replace the block that entered the cache earliest.
LRU	Replace the least recently used block.
LFU	Replace the least frequently used block.
Random	Choose a block randomly.

Direct-mapped caches do not need a replacement choice: the mapping already determines the single cache line that must be used.

Write Policies

Policy	Behavior
Write-through	Update the cache and main memory on each write. Main memory stays current, but traffic is higher.
Write-back	Update the cache first. A dirty block is written to main memory when it is replaced.

10. Virtual Memory

Virtual memory gives a process a logical address space that is translated to physical memory. Paging is a common implementation technique.

Virtual Address → Page Table / MMU → Physical Address

Term	Meaning
Virtual address	Address used by a program.
Physical address	Actual location in RAM.
Page table	Maps virtual pages to physical page frames.
Page fault	Required page is not currently mapped as present; it may require loading from secondary storage.

11. Formula Summary

Topic	Formula
Number of patterns with n bits	2^n
Unsigned range	0 to 2^n − 1
Two's-complement range	−2^(n−1) to 2^(n−1) − 1
Address bits	ceil(log2(number of addressable locations))
Clock period	T = 1/f
Clock frequency	f = 1/T
CPU cycles	Instruction Count × CPI
CPU time	(Instruction Count × CPI) / Clock Rate
CPI	(CPU time × Clock Rate) / Instruction Count
Speedup	Old time / New time
Pipeline time	(k + n − 1) × t
Pipeline time with stalls	(k + n − 1 + s) × t
Main-memory blocks	Main Memory Size / Block Size

Offset bits	log2(Block Size)
Direct-map index bits	log2(Cache Lines)
Direct-map tag bits	Address Bits − Index Bits − Offset Bits
Memory block number	floor(Address / Block Size)
Direct-map cache index	Memory Block mod Cache Lines
Block offset	Address mod Block Size
Set count	Cache Lines / Ways
Set number	Memory Block mod Number of Sets
Hit rate	Hits / Accesses
Miss rate	1 − Hit Rate
Indexed addressing	EA = Base + Index
Array addressing	EA = Base + (Index × Element Size)

12. Learning Checklist

- Explain the difference between computer architecture and computer organization.
- Identify the roles of the CPU, ALU, control unit, registers, buses, memory, and I/O.
- Convert between binary and hexadecimal and interpret bit patterns correctly.
- Represent and decode negative integers using two's complement.
- Calculate the number of address bits required for byte-addressable or word-addressable memory.
- Explain MARIE register roles and the 4-bit opcode + 12-bit address instruction format.
- Trace LOAD, STORE, ADD, SUBT, INPUT, OUTPUT, and HALT operations through the AC and memory.
- Decode a MARIE instruction and trace its fetch, decode, and execute cycle using the appropriate registers.
- Build and use a simple symbol table.
- Distinguish immediate, register, direct, indirect, and indexed addressing.
- Use instruction count, CPI, clock rate, and execution time formulas.
- Calculate pipeline time and speedup and identify structural, data, and control hazards.
- Explain memory hierarchy, locality, SRAM vs. DRAM, latency, and bandwidth.
- Calculate cache block number, offset, index/set, tag, and direct-mapped placement.
- Explain direct-mapped, fully associative, and set-associative caches.
- Explain write-through, write-back, replacement policies, and virtual memory basics.